Cahier d'exercices Python — du C à l'automatisation cyber

Un parcours progressif d'**137 exercices**, pensé pour quelqu'un qui sait déjà programmer (C) et veut devenir très bon en Python, surtout pour **automatiser des tâches** (cyber, sysadmin, data).

Comment utiliser ce cahier

- **Rythme conseillé** : 30 à 60 min par jour. Un niveau = environ 1 à 2 semaines. Le parcours complet représente 4 à 6 mois à ce rythme, sans pression.
- **Un dossier par niveau** (`niveau01/`, `niveau02/`…) et **un fichier par exercice** (`ex01_03.py`). Tout dans un dépôt Git, avec un commit par exercice. Tu auras un historique de progression et l'habitude de Git.
- **Cherche avant de regarder la solution.** Bloqué plus de 20 minutes : relis les notes du niveau, la doc officielle, puis seulement après demande de l'aide.
- **Réécris ton code une deuxième fois** quand il marche : plus court, plus lisible, plus « pythonique ». C'est là que tu progresses le plus.
- **Coche** les cases `[ ]` au fur et à mesure.
- Symboles : ★ facile · ★★ moyen · ★★★ difficile · **[cyber]** orienté cyber · ◆ mini-projet de fin de niveau.

⚠ **Éthique** : tous les exercices réseau/sécurité se font **uniquement sur ta machine (`127.0.0.1`), ton propre labo, ou des plateformes prévues pour ça** (TryHackMe, HackTheBox, CTF). Jamais sur des systèmes sans autorisation écrite.

Sommaire

Niveau	Thème	Exercices
0	Mise en place	5
1	Bases : variables, types, I/O	10
2	Conditions et boucles	10
3	Fonctions	10
4	Structures de données	12
5	Fichiers, erreurs, modules	10
6	Chaînes et regex	10
7	Programmation orientée objet	10
8	Python idiomatique	10
9	Automatisation système	10
10	Réseau et web **[cyber]**	10
11	Concurrence et performance	8
12	Cyber avancé **[cyber]**	10
13	Qualité, tests, packaging	6
14	Projets finaux	6

Niveau 0 — Mise en place

Objectif : un environnement propre dès le début.

- ☐ **0.1** ★ Installe Python 3.12+ et vérifie avec `python3 --version`. Lance le REPL et calcule `2**100`. Note ce qui te frappe par rapport au C (indice : taille des entiers).
- ☐ **0.2** ★ Crée un environnement virtuel (`python3 -m venv .venv`), active-le, puis installe `requests` avec `pip`. Vérifie avec `pip list`.
- ☐ **0.3** ★ Crée un dépôt Git pour ce cahier, avec un `.gitignore` qui exclut `.venv/` et `__pycache__/`.
- ☐ **0.4** ★ Écris un `hello.py` qui affiche « Bonjour » et exécute-le de deux façons : `python3 hello.py` et en le rendant exécutable avec un shebang (`#!/usr/bin/env python3`).
- ☐ **0.5** ★ Dans le REPL, utilise `help(str)`, `dir(str)` et `type()` pour explorer. Trouve 3 méthodes de `str` que tu ne connaissais pas.

Niveau 1 — Bases : variables, types, entrées/sorties

À retenir venant du C : pas de déclaration de type, pas de `;`, l'indentation remplace les `{}`, les entiers n'ont pas de limite, `/` renvoie toujours un float (`//` pour la division entière), pas de `++`.

- ☐ **1.1** ★ Demande le prénom et l'âge de l'utilisateur, affiche « Salut X, dans 10 ans tu auras Y ans ». Utilise une f-string.
- ☐ **1.2** ★ Convertisseur Celsius ↔ Fahrenheit. Affiche le résultat avec 1 décimale (`f"{x:.1f}"`).
- ☐ **1.3** ★ Calcule l'aire et le périmètre d'un cercle avec `math.pi`.
- ☐ **1.4** ★ Divise deux nombres saisis par l'utilisateur et affiche le quotient entier, le reste, et le résultat décimal (`//`, `%`, `/`).
- ☐ **1.5** ★ Affiche `type()` de `5`, `5.0`, `"5"`, `True`, `None`, `[5]`, `(5,)`, `{5}`, `{5: 5}`. Prédis chaque résultat avant d'exécuter.
- ☐ **1.6** ★ Échange deux variables sans variable temporaire (`a, b = b, a`). Explique pourquoi c'est possible.
- ☐ **1.7** ★★ Convertis un nombre en binaire, octal et hexadécimal avec `bin()`, `oct()`, `hex()` **puis** avec le formatage (`f"{n:08b}"`, `f"{n:#x}"`).
- ☐ **1.8** ★★ Demande un nombre de secondes, affiche-le en `hh:mm:ss` (utilise `divmod`).
- ☐ **1.9** ★★ Teste `0.1 + 0.2 == 0.3`. Explique le résultat (tu connais déjà l'IEEE 754 du C). Trouve deux façons correctes de comparer : `math.isclose` et le module `decimal`.
- ☐ **1.10** ★★ ◆ **Mini-calculatrice** : lit une expression du type `12 * 3`, découpe avec `split()`, calcule, et gère `+ - * / **`. Pas de `eval` !

Niveau 2 — Conditions et boucles

À retenir : `if/elif/else`, `for x in iterable` (pas de compteur manuel), `while`, `break`/`continue`, et le `else` sur les boucles. Pense à `range()`, `enumerate()`, `zip()`.

- ☐ **2.1** ★ Affiche si un nombre est pair ou impair, positif, négatif ou nul.
- ☐ **2.2** ★ Affiche la table de multiplication d'un nombre donné, de 1 à 10.
- ☐ **2.3** ★ FizzBuzz de 1 à 100.
- ☐ **2.4** ★ Somme des nombres de 1 à N, puis vérifie avec la formule `N(N+1)/2` et avec `sum(range(...))`.
- ☐ **2.5** ★ Jeu du « plus ou moins » : l'ordinateur choisit un nombre entre 1 et 100 (`random.randint`), l'utilisateur devine, avec indices et compteur d'essais.
- ☐ **2.6** ★★ Affiche un triangle de `*` de hauteur N, puis un sapin (triangle centré).
- ☐ **2.7** ★★ Liste tous les nombres premiers jusqu'à N. Version naïve d'abord, puis crible d'Ératosthène.
- ☐ **2.8** ★★ Suite de Fibonacci : les N premiers termes, en boucle puis en générateur (aperçu, tu verras `yield` plus tard).
- ☐ **2.9** ★★ Conjecture de Syracuse : pour un nombre de départ, affiche la suite jusqu'à 1 et le nombre d'étapes. Trouve le nombre < 10 000 avec la plus longue suite.
- ☐ **2.10** ★★ ◆ **Pierre-Feuille-Ciseaux** contre l'ordinateur, en 3 manches gagnantes, avec score affiché.

Niveau 3 — Fonctions

À retenir : arguments par défaut, arguments nommés, `*args`, `**kwargs`, retour de plusieurs valeurs (tuple), docstrings, type hints (`def f(x: int) -> int:`). Attention au piège des **valeurs par défaut mutables**.

- ☐ **3.1** ★ Écris `est_premier(n)` et réutilise-la pour afficher les premiers < 100.
- ☐ **3.2** ★ `factorielle(n)` en itératif, puis en récursif. Compare avec `math.factorial`.
- ☐ **3.3** ★ `pgcd(a, b)` (Euclide) et `ppcm(a, b)`.
- ☐ **3.4** ★★ Fonction `stats(nombres)` qui renvoie min, max, moyenne et médiane **sans** utiliser le module `statistics`.
- ☐ **3.5** ★★ Fonction `saluer(nom, langue="fr", *, majuscules=False)`. Appelle-la de 4 façons différentes.
- ☐ **3.6** ★★ Fonction `somme(*args)` puis `afficher(**kwargs)` qui imprime `clé = valeur`. Dépaquette une liste et un dict à l'appel avec `*` et `**`.
- ☐ **3.7** ★★ Reproduis le piège : `def ajouter(x, liste=[])`. Appelle-la 3 fois et explique. Corrige avec `None`.
- ☐ **3.8** ★★ Tours de Hanoï récursif : affiche les déplacements pour N disques.
- ☐ **3.9** ★★★ Écris un décorateur `@chrono` qui affiche le temps d'exécution d'une fonction (`time.perf_counter`, `functools.wraps`). Applique-le à Fibonacci récursif, puis ajoute `@functools.lru_cache` et compare.
- ☐ **3.10** ★★★ ◆ **Mots de passe** : `generer_mdp(longueur, majuscules, chiffres, symboles)` avec le module `secrets` (pas `random`, cherche pourquoi), et `force_mdp(mdp)` qui note le mot de passe sur 5.

Niveau 4 — Structures de données

À retenir : c'est ici que Python devient puissant. `list`, `tuple`, `dict`, `set`, slicing, compréhensions, tri avec `key=`, `collections` (`Counter`, `defaultdict`, `deque`).

- ☐ **4.1** ★ Liste de N entiers : affiche le max, le min, la somme, la liste inversée (3 façons pour inverser).
- ☐ **4.2** ★ Slicing : à partir de `list(range(20))`, extrais les 5 premiers, les 5 derniers, les indices pairs, la liste inversée, un élément sur 3 en partant de la fin.
- ☐ **4.3** ★ Supprime les doublons d'une liste en gardant l'ordre. Deux méthodes : boucle, puis `dict.fromkeys`.
- ☐ **4.4** ★★ Compréhensions : carrés des nombres pairs de 1 à 50, dict `{nombre: carré}`, set des lettres d'une phrase.
- ☐ **4.5** ★★ Compte la fréquence de chaque mot d'un texte, d'abord avec un `dict` à la main, puis avec `Counter`. Affiche le top 5.
- ☐ **4.6** ★★ Carnet d'adresses en dict de dicts : ajouter, rechercher, modifier, supprimer un contact.
- ☐ **4.7** ★★ Opérations sur sets : à partir de deux listes, trouve les éléments communs, uniques à chacune, et l'union.
- ☐ **4.8** ★★ Trie une liste de tuples `(nom, age, note)` par note décroissante puis par nom croissant (`sorted` avec `key=`).
- ☐ **4.9** ★★ Anagrammes : regroupe une liste de mots par anagrammes (`defaultdict(list)`, clé = lettres triées).
- ☐ **4.10** ★★★ Transpose une matrice (liste de listes) avec `zip(*m)`, puis multiplie deux matrices sans bibliothèque.
- ☐ **4.11** ★★★ Implémente une **pile** et une **file** avec `list` puis `collections.deque`. Utilise la pile pour vérifier l'équilibrage des parenthèses `()[]{}`.
- ☐ **4.12** ★★★ ◆ **Gestionnaire de tâches** en console : ajouter, lister, marquer fait, supprimer, filtrer par priorité. Les données sont en mémoire pour l'instant (la persistance viendra au niveau 5).

Niveau 5 — Fichiers, erreurs, modules

À retenir : `with open(...)`, `pathlib.Path`, `try/except/else/finally`, exceptions personnalisées, `json`, `csv`, `import`, `if __name__ == "__main__":`, `argparse`, `logging`.

- ☐ **5.1** ★ Lis un fichier texte, affiche le nombre de lignes, de mots et de caractères (comme `wc`).
- ☐ **5.2** ★ Écris un programme qui copie un fichier ligne par ligne en numérotant les lignes.
- ☐ **5.3** ★★ Demande un nombre à l'utilisateur en boucle jusqu'à ce que la saisie soit valide (`try/except ValueError`).
- ☐ **5.4** ★★ Lis et écris du JSON : sauvegarde le gestionnaire de tâches du 4.12 dans `taches.json` et recharge-le au démarrage.
- ☐ **5.5** ★★ Lis un CSV avec `csv.DictReader`, calcule des statistiques par colonne (moyenne, max), écris un nouveau CSV filtré.
- ☐ **5.6** ★★ Reprends 5.1 avec `pathlib` : parcours un dossier récursivement (`rglob("*.py")`) et affiche la taille totale par extension.
- ☐ **5.7** ★★ Crée une exception personnalisée `SoldeInsuffisant` et utilise-la dans une petite fonction `retirer(solde, montant)`. Utilise `raise ... from ...` dans un cas.
- ☐ **5.8** ★★★ Transforme le 5.1 en **vrai outil en ligne de commande** avec `argparse` : `mon_wc.py fichier.txt --lignes --mots`. Ajoute `--help`.
- ☐ **5.9** ★★★ Ajoute du `logging` (niveaux DEBUG/INFO/ERROR, écriture dans un fichier, format avec horodatage) à un de tes scripts précédents. Abandonne les `print` de débogage.
- ☐ **5.10** ★★★ ◆ **Mini base de données** de notes : un module `notes.py` (fonctions) importé par un script CLI `main.py` avec sous-commandes (`add`, `list`, `search`, `delete`), stockage JSON, gestion d'erreurs propre.

Niveau 6 — Chaînes de caractères et expressions régulières

À retenir : c'est le cœur de l'automatisation et de l'analyse de logs. Méthodes `str`, `split`/`join`, `strip`, `replace`, `startswith`, f-strings avancées, module `re` (`search`, `match`, `findall`, `finditer`, `sub`, groupes nommés, `re.compile`, flags).

- ☐ **6.1** ★ Inverse une chaîne, teste si c'est un palindrome (en ignorant casse et espaces).
- ☐ **6.2** ★ Compte les voyelles, consonnes, chiffres et espaces d'une chaîne.
- ☐ **6.3** ★★ Chiffrement de César : chiffre/déchiffre avec un décalage. Puis casse-le par force brute en testant les 25 décalages.
- ☐ **6.4** ★★ Écris une fonction qui valide une adresse email avec une regex simple, puis une qui extrait **toutes** les adresses email d'un texte.
- ☐ **6.5** ★★ Extrais toutes les adresses IPv4 d'un texte avec `re.findall`. Puis valide que chaque octet est ≤ 255 (la regex seule ne suffit pas : utilise le module `ipaddress`).
- ☐ **6.6** ★★ Convertis `snake_case` ↔ `camelCase` avec `re.sub`.
- ☐ **6.7** ★★★ Parse cette ligne de log avec des **groupes nommés** : `192.168.1.10 - - [21/Sep/2026:10:15:32 +0200] "GET /index.php HTTP/1.1" 200 1543`. Renvoie un dict (ip, date, méthode, chemin, code, taille).
- ☐ **6.8** ★★★ **[cyber]** Génère un faux fichier de log (ou utilise un vrai `auth.log` de ton labo). Compte les lignes contenant `Failed password` et affiche le top 10 des IP sources.
- ☐ **6.9** ★★★ **[cyber]** Décode et encode : base64, hexadécimal, URL-encoding, ROT13, en utilisant les modules standard (`base64`, `binascii`, `urllib.parse`, `codecs`). Puis écris un petit « CyberChef » en ligne de commande qui enchaîne plusieurs décodages.
- ☐ **6.10** ★★★ ◆ **Analyseur de logs** : script `logstat.py` qui prend un fichier de log web en argument et sort : nombre de requêtes, top IP, top URL, répartition des codes HTTP, heures les plus actives.

Niveau 7 — Programmation orientée objet

À retenir : `class`, `__init__`, `self`, attributs de classe vs d'instance, méthodes spéciales (`__str__`, `__repr__`, `__eq__`, `__len__`, `__iter__`), `@property`, `@classmethod`, `@staticmethod`, héritage, `super()`, `@dataclass`, classes abstraites (`abc`).

- ☐ **7.1** ★ Classe `Point` avec `x`, `y`, méthode `distance_a(autre)`, et `__str__`.
- ☐ **7.2** ★★ Classe `CompteBancaire` : dépôt, retrait (avec ton exception `SoldeInsuffisant`), historique des opérations, `@property` pour le solde.
- ☐ **7.3** ★★ Classe `Vecteur` avec surcharge des opérateurs : `+`, `-`, `*` (scalaire), `==`, `abs()`, `len()`, `__repr__`.
- ☐ **7.4** ★★ Héritage : `Animal` → `Chien`, `Chat`, avec polymorphisme sur une méthode `parler()`. Utilise `super()`.
- ☐ **7.5** ★★ Refais le 7.1 et le carnet du 4.6 avec `@dataclass`. Compare avec l'écriture à la main.
- ☐ **7.6** ★★★ Classe abstraite `Forme` (`abc.ABC`, `@abstractmethod aire()`), avec `Cercle`, `Rectangle`, `Triangle`. Trie une liste de formes par aire.
- ☐ **7.7** ★★★ Classe `Pile` itérable (`__iter__`, `__len__`, `__contains__`, `__bool__`) et utilisable dans un `for`.
- ☐ **7.8** ★★★ Gestionnaire de contexte : classe `Chrono` avec `__enter__`/`__exit__` utilisable comme `with Chrono(): ...`. Puis refais-le avec `contextlib.contextmanager`.
- ☐ **7.9** ★★★ Composition vs héritage : modélise une bibliothèque (`Livre`, `Membre`, `Bibliotheque`) et justifie tes choix en 3 lignes en commentaire.
- ☐ **7.10** ★★★ ◆ **Inventaire d'assets [cyber]** : classes `Machine`, `Service`, `Vulnerabilite` (avec score CVSS), `Inventaire` ; export JSON/CSV, tri et filtres par criticité. C'est la base d'un futur outil de gestion de risques.

Niveau 8 — Python idiomatique (« pythonique »)

À retenir : itérateurs, générateurs (`yield`), `itertools`, `functools`, `map/filter`, lambdas, unpacking avancé, `zip`, `any`/`all`, walrus `:=`, `match/case`, EAFP vs LBYL, `typing`.

- ☐ **8.1** ★★ Réécris 5 de tes anciens exercices en version plus courte avec compréhensions, `sum`, `any`, `all`, `enumerate`, `zip`.
- ☐ **8.2** ★★ Écris un générateur `lire_lignes(fichier)` qui `yield` chaque ligne nettoyée, et traite un gros fichier (génère-en un de 1 million de lignes) en vérifiant que la mémoire reste faible.
- ☐ **8.3** ★★ Générateur infini `premiers()`, puis récupère les 20 premiers avec `itertools.islice`.
- ☐ **8.4** ★★★ Utilise `itertools` : `product`, `permutations`, `combinations`, `chain`, `groupby`. Génère toutes les combinaisons de 4 chiffres (code PIN) et compte-les.
- ☐ **8.5** ★★★ Implémente `map`, `filter` et `reduce` toi-même, puis utilise `functools.reduce` pour calculer un produit.
- ☐ **8.6** ★★★ Écris un **itérateur** en classe (`__iter__` + `__next__`) qui parcourt une plage de dates jour par jour.
- ☐ **8.7** ★★★ Utilise `match/case` pour écrire un petit interpréteur de commandes (`"add 5"`, `"list"`, `"quit"`, ...).
- ☐ **8.8** ★★★ Ajoute des **type hints** complets à ton `logstat.py` et vérifie avec `mypy`. Corrige les erreurs remontées.
- ☐ **8.9** ★★★ Décorateur avec paramètres : `@retry(tentatives=3, delai=1)` qui rejoue une fonction en cas d'exception (exponentiel en bonus). Sera très utile pour le réseau.
- ☐ **8.10** ★★★ ◆ **Pipeline de données** : chaîne de générateurs (lire → parser → filtrer → agréger → écrire) sur un gros fichier de logs, sans jamais tout charger en mémoire.

Niveau 9 — Automatisation système

À retenir : `os`, `sys`, `pathlib`, `shutil`, `subprocess`, `glob`, `hashlib`, `datetime`, `zipfile`/`tarfile`, `sqlite3`, variables d'environnement, planification (cron / `schedule`).

- ☐ **9.1** ★★ **Range-tout** : trie les fichiers d'un dossier en sous-dossiers par extension (`Images/`, `Docs/`…). Mode `--dry-run` obligatoire avant de vraiment déplacer.
- ☐ **9.2** ★★ **Renommeur en masse** : renomme des fichiers selon un motif (préfixe, numérotation, date de modification). Annulation possible.
- ☐ **9.3** ★★ Détecte les **doublons** dans un dossier : compare d'abord par taille, puis par hash SHA-256 (`hashlib`, lecture par blocs).
- ☐ **9.4** ★★ Lance des commandes système avec `subprocess.run` (`ls`, `ping -c 1`, `whoami`), récupère stdout/stderr/code de retour. Lis pourquoi `shell=True` est dangereux avant de l'utiliser.
- ☐ **9.5** ★★★ Script de **sauvegarde** : archive `.tar.gz` d'un dossier avec date dans le nom, garde seulement les 5 dernières sauvegardes.
- ☐ **9.6** ★★★ Surveille un dossier : détecte les fichiers créés/modifiés/supprimés en comparant des snapshots (taille + date + hash), puis avec `watchdog`.
- ☐ **9.7** ★★★ Stocke des résultats dans **SQLite** (`sqlite3`) : crée une table, insère, requête, avec des requêtes paramétrées (`?`). Comprends pourquoi on ne concatène jamais une entrée utilisateur dans du SQL.
- ☐ **9.8** ★★★ **[cyber] Vérificateur d'intégrité (mini-FIM)** : calcule les hash de fichiers sensibles (`/etc/passwd`, etc.), les stocke en base, et signale toute modification lors d'un second passage.
- ☐ **9.9** ★★★ **[cyber]** Récupère la liste des processus, des ports en écoute et des utilisateurs connectés avec `psutil`. Exporte un rapport de l'état de la machine en JSON.
- ☐ **9.10** ★★★ ◆ **Rapport quotidien** : script qui collecte des infos système (disque, RAM, derniers échecs de connexion, services en échec), génère un rapport HTML ou Markdown horodaté, et se lance automatiquement chaque jour (cron ou `schedule`).

Niveau 10 — Réseau et web [cyber]

À retenir : `socket`, `requests`, `http.server`, `urllib`, `json`, BeautifulSoup, `ipaddress`, APIs REST, sessions et cookies, timeouts.

> À faire sur `127.0.0.1`, ton propre labo, ou sur des cibles explicitement autorisées (machines TryHackMe/HTB, sites d'entraînement).

- ☐ **10.1** ★★ Requête `GET` avec `requests` : affiche code, en-têtes, taille et 200 premiers caractères du corps. Toujours avec un `timeout`.
- ☐ **10.2** ★★ Interroge une API publique (ex. une API météo ou de blagues), parse le JSON, affiche un résultat lisible. Gère erreurs réseau et codes ≠ 200.
- ☐ **10.3** ★★ Serveur **TCP « echo »** avec `socket` (bind/listen/accept/recv/send), puis un client. Tu retrouveras la logique du C.
- ☐ **10.4** ★★★ Petit **chat** TCP multi-clients (d'abord avec `select`, en bonus avec des threads).
- ☐ **10.5** ★★★ Scanner de **ports TCP** simple (`connect_ex`) sur `127.0.0.1`, avec timeout et plage de ports en paramètre. Affiche le service probable avec `socket.getservbyport`.
- ☐ **10.6** ★★★ **Banner grabbing** : pour chaque port ouvert, lis la bannière du service et affiche-la.
- ☐ **10.7** ★★★ Vérifie les **en-têtes de sécurité** d'un site (autorisé) : `Strict-Transport-Security`, `Content-Security-Policy`, `X-Frame-Options`, etc. Produis un rapport avec note.
- ☐ **10.8** ★★★ Scraper avec `requests` + `BeautifulSoup` : extrait tous les liens d'une page, classe-les en internes/externes, détecte les liens cassés.
- ☐ **10.9** ★★★ Crée une petite **API REST** avec Flask ou FastAPI (`/notes` GET/POST), puis un client Python qui la consomme.
- ☐ **10.10** ★★★ ◆ **Recon toolkit** : un seul script CLI (`argparse`, sous-commandes) qui fait : résolution DNS, scan de ports, banner grabbing, en-têtes HTTP. Sortie JSON. Uniquement pour des cibles de labo.

Niveau 11 — Concurrence et performance

À retenir : le **GIL**, `threading` (I/O), `multiprocessing` (CPU), `concurrent.futures`, `asyncio`/`aiohttp`, `queue`, profilage (`cProfile`, `timeit`).

- ☐ **11.1** ★★ Télécharge 20 URLs en séquentiel puis avec `ThreadPoolExecutor`. Compare les temps.
- ☐ **11.2** ★★ Calcul CPU lourd (ex. nombres premiers jusqu'à 5 millions) : séquentiel, `threading`, puis `multiprocessing`. Explique les résultats en lien avec le GIL.
- ☐ **11.3** ★★★ Reprends ton scanner de ports (10.5) en version **multi-thread**. Mesure le gain, limite le nombre de threads.
- ☐ **11.4** ★★★ Modèle producteur/consommateur avec `queue.Queue` et plusieurs threads. Termine proprement (sentinelle).
- ☐ **11.5** ★★★ Écris un exemple simple d'`asyncio` (`async`/`await`, `gather`), puis refais le scanner de ports en **asynchrone** (`asyncio.open_connection`).
- ☐ **11.6** ★★★ Provoque et corrige une **race condition** avec un compteur partagé et un `threading.Lock`.
- ☐ **11.7** ★★★ Profile un de tes scripts lents avec `cProfile` et `timeit`, trouve le goulot d'étranglement, optimise, et prouve le gain par des chiffres.
- ☐ **11.8** ★★★ ◆ **Scanner haute performance** : scanner asynchrone avec limite de concurrence (`Semaphore`), barre de progression, export des résultats, reprise après interruption.

Niveau 12 — Cyber avancé [cyber]

À retenir : cryptographie (`hashlib`, `hmac`, `secrets`, `cryptography`), manipulation d'octets (`bytes`, `struct`), `scapy`, `pwntools`, analyse de fichiers, parsing de formats binaires.

> Exercices à faire dans un cadre légal : CTF, labos, TryHackMe, tes propres fichiers et machines.

- ☐ **12.1** ★★ Manipule des **octets** : convertis chaîne ↔ `bytes`, fais un XOR entre deux buffers, encode/décode en hex. Résous un mini-défi « XOR à clé d'un octet » (force brute sur 256 clés + détection de texte lisible).
- ☐ **12.2** ★★ Calcule MD5, SHA-1, SHA-256 d'un fichier (par blocs). Compare avec `sha256sum`. Explique pourquoi MD5 et SHA-1 sont déconseillés pour la sécurité.
- ☐ **12.3** ★★★ Stocke des mots de passe correctement : sel aléatoire + `hashlib.scrypt` ou `pbkdf2_hmac` (ou `argon2-cffi`). Implémente `hasher()` et `verifier()` avec comparaison à temps constant (`hmac.compare_digest`).
- ☐ **12.4** ★★★ Chiffrement symétrique avec la lib `cryptography` (Fernet, puis AES-GCM). Chiffre/déchiffre un fichier. Note pourquoi on n'invente jamais sa propre crypto.
- ☐ **12.5** ★★★ **Vigenère** : chiffre, déchiffre, puis casse-le avec une analyse de fréquences (test de Kasiski ou indice de coïncidence).
- ☐ **12.6** ★★★ Lis l'en-tête d'un fichier binaire avec `struct` (ex. PNG ou ELF) : vérifie la signature magique, affiche les champs. Écris un mini « `file` » qui identifie 10 types par magic bytes.
- ☐ **12.7** ★★★ Parseur de logs de sécurité : détecte des motifs de **force brute SSH** (N échecs en T secondes depuis une IP) et génère une liste d'alertes.
- ☐ **12.8** ★★★ Résous 5 défis de type CTF « crypto/forensics » faciles (TryHackMe ou picoCTF) **uniquement avec des scripts Python** que tu écris toi-même. Garde un dossier `writeups/` avec une note par défi.
- ☐ **12.9** ★★★ Avec `scapy` sur ton labo : forge et envoie un paquet ICMP, sniffe quelques paquets, extrais les IP/ports dans un rapport. (Alternative : lis un `.pcap` avec `scapy.rdpcap` et fais des stats.)
- ☐ **12.10** ★★★ ◆ **Détecteur d'intrusion minimal** : lit un `.pcap` ou un log, applique des règles (fichier YAML/JSON de signatures), génère des alertes horodatées avec niveau de gravité.

Niveau 13 — Qualité, tests, packaging

À retenir : c'est ce qui sépare un script jetable d'un vrai outil. `pytest`, `unittest.mock`, `ruff`/`black`, `mypy`, structure de projet, `pyproject.toml`, documentation, CI.

- ☐ **13.1** ★★ Écris des **tests** `pytest` pour tes fonctions du niveau 3 et 6 (cas normaux, limites, erreurs avec `pytest.raises`).
- ☐ **13.2** ★★★ Teste du code qui touche au réseau ou aux fichiers avec des **mocks** (`unittest.mock`, fixtures `tmp_path`) sans jamais faire de vraie requête.
- ☐ **13.3** ★★★ Configure `ruff` (lint + format) et `mypy` sur un de tes projets, corrige tout, ajoute un hook `pre-commit`.
- ☐ **13.4** ★★★ Restructure ton projet en **package** : `src/monoutil/`, `tests/`, `pyproject.toml`, point d'entrée console (`[project.scripts]`). Installe-le avec `pip install -e .`.
- ☐ **13.5** ★★★ Rédige un `README.md` clair (installation, usage, exemples, avertissement légal) et une doc de tes fonctions par docstrings.
- ☐ **13.6** ★★★ ◆ Mets en place une **CI** GitHub Actions qui lance lint, mypy et tests à chaque push.

Niveau 14 — Projets finaux

Choisis-en **au moins 3**. Chacun doit avoir : dépôt Git propre, README, tests, gestion d'erreurs, logs, CLI avec `argparse`.

- ☐ **14.1 [cyber] Mini-SIEM** : ingère des logs de plusieurs sources (auth, web, système), normalise, stocke dans SQLite, applique des règles de détection, génère un tableau de bord HTML.
- ☐ **14.2 [cyber] Scanner de vulnérabilités léger** : découvre les services d'un labo, identifie leurs versions, les compare à une base locale de CVE (JSON), génère un rapport priorisé.
- ☐ **14.3 [cyber] Gestionnaire de mots de passe** en CLI : coffre chiffré (AES-GCM + dérivation de clé), génération, recherche, copie dans le presse-papiers avec effacement automatique.
- ☐ **14.4 Bot d'automatisation personnel** : surveille des sources (flux RSS, API, dossier), déclenche des actions (notifications, archivage, résumé quotidien par e-mail ou Discord).
- ☐ **14.5 [cyber] Honeypot simple** : faux service (SSH ou HTTP) sur ton labo qui journalise les tentatives de connexion, agrège les IP, produit des statistiques.
- ☐ **14.6 Application au choix** : une vraie utilité pour toi (suivi de candidatures, gestion de cours, outil de révision…). L'important est de la finir et de l'utiliser.

Suivi de progression

Niveau	Commencé le	Terminé le	Ce que j'ai appris / ce qui m'a bloqué
0			
1			
2			
3			
4			
5			
6			
7			
8			
9			
10			
11			
12			
13			
14			

Ressources utiles

- **Doc officielle** : docs.python.org/3 (le tutoriel officiel et la référence de la bibliothèque standard)
- **Pythonic** : PEP 8 (style), PEP 20 (`import this`), *Fluent Python* de Luciano Ramalho (pour le niveau 8 et au-delà)
- **Cyber avec Python** : *Black Hat Python* (Justin Seitz), *Violent Python*, *Automate the Boring Stuff* (gratuit en ligne) pour l'automatisation générale
- **Pratique** : TryHackMe (parcours de scripting), picoCTF, HackTheBox, Advent of Code (excellent pour l'algorithmique en Python)

Conseils pour tenir sur la durée

1. **Régularité avant intensité** : 30 min tous les jours valent mieux que 5 h le dimanche.
2. **Tape le code, ne copie jamais.** Même quand tu t'inspires d'une solution.
3. **Lis du code des autres** : explore la bibliothèque standard et des petits projets open source.
4. **Refais les exercices anciens** de temps en temps, sans regarder ta solution : c'est la meilleure révision.
5. **Garde un fichier** `notes.md` avec les pièges rencontrés (comme le `list=[]` par défaut). Ces pièges reviennent.